



LeetCode 704: Binary Search

1. Problem Title & Link

Title: LeetCode 704: Binary Search

Link: <https://leetcode.com/problems/binary-search/>

2. Problem Statement (Short Summary)

Given:

- A **sorted** array `nums`
- A target value `target`

Return the index of target.

If the target does not exist, return:

-1

Example

Input:

`nums = [-1,0,3,5,9,12]`

`target = 9`

Output:

4

Because:

`nums[4] = 9`

3. Examples (Input → Output)

Example 1

Input:

`nums = [-1,0,3,5,9,12]`

`target = 9`

Output:

4

Example 2

Input:

`nums = [-1,0,3,5,9,12]`

`target = 2`



Output:

-1

Example 3

Input:

nums = [5]

target = 5

Output:

0

Example 4

Input:

nums = [1,3,5,7,9]

target = 1

Output:

0

4. Constraints

$1 \leq \text{nums.length} \leq 10^4$

$-10^4 < \text{nums}[i], \text{target} < 10^4$

Important:

nums is sorted in ascending order.

Without sorting:

Binary Search does NOT work.

5. Core Concept (Pattern / Topic)

Binary Search

Secondary Topics:

- Divide and Conquer
- Search Space Reduction

6. Thought Process (Step-by-Step Explanation)

Brute Force Idea

Scan every element.



```
for i in range(len(nums)):
    if nums[i] == target:
        return i
```

Complexity

$O(n)$

For:

1,000,000 elements

this becomes expensive.

Key Observation

Array is already sorted.

Example:

`[-1,0,3,5,9,12]`

Instead of checking every element:

Check the middle element.

Example

Target:

9

Middle:

5

Compare:

$9 > 5$

Since array is sorted:

Target can only exist on the right side.

Discard left half.

New search space:

`[9,12]`

Middle:

9

Found.

Why Binary Search Is Fast

Every comparison removes half of the array.

Example:



1000 elements

↓

500

↓

250

↓

125

↓

62

↓

31

↓

15

↓

7

↓

3

↓

1

Only about:

$\log_2(n)$

comparisons.

7. Visual / Intuition Diagram (ASCII Diagram)

```
Input:  
[-1, 0, 3, 5, 9, 12]
```

```
target = 9  
Initial:
```



```

L           M           R

-1 0 3 5 9 12
      ↑
      5

Compare:
9 > 5
Discard left half.
           L     M     R

-1 0 3 5 9 12
      ↑
      9

Found.

```

8. Pseudocode (Language Independent)

```

left = 0

right = n-1

while left <= right

    mid = (left + right) / 2

    if nums[mid] == target

        return mid

    else if target > nums[mid]

        left = mid + 1

    else

        right = mid - 1

return -1

```

9. Code Implementation

Python

```

class Solution:
    def search(self, nums, target):

        left = 0
        right = len(nums) - 1

```



```
while left <= right:

    mid = (left + right) // 2

    if nums[mid] == target:
        return mid

    elif nums[mid] < target:
        left = mid + 1

    else:
        right = mid - 1

return -1
```

Java

```
class Solution {

    public int search(int[] nums, int target) {

        int left = 0;
        int right = nums.length - 1;

        while(left <= right) {

            int mid =
                left + (right - left) / 2;

            if(nums[mid] == target)
                return mid;

            else if(nums[mid] < target)
                left = mid + 1;

            else
                right = mid - 1;

        }

        return -1;

    }

}
```

10. Time & Space Complexity

Time Complexity

$O(\log n)$

Search space halves every iteration.

**Space Complexity**

$O(1)$

Only a few variables are used.

11. Common Mistakes / Edge Cases**Mistake 1**

Using Binary Search on an unsorted array.

Example:

[5,1,8,2]

Binary Search becomes invalid.

Mistake 2

Wrong loop condition.

Correct:

while left <= right

Not:

while left < right

for this implementation.

Mistake 3

Updating wrong boundary.

Wrong:

left = mid

Can cause infinite loops.

Correct:

left = mid + 1

Edge Cases**Single Element Found**

Input:

[5]

Target:

5

Output:

0



Single Element Not Found

Input:

[5]

Target:

1

Output:

-1

12. Detailed Dry Run

Input:

nums = [-1,0,3,5,9,12]

target = 9

Initial:

left = 0

right = 5

left	right	mid	nums[mid]	Action
0	5	2	3	Go Right
3	5	4	9	Found

Return:

4

Another Example

Input:

target = 2

left	right	mid	value
0	5	2	3
0	1	0	-1
1	1	1	0

Not found.

Return:

-1

13. Common Use Cases (Real-Life / Interview)



Dictionary Search

Finding a word in a sorted dictionary.

Database Index Lookup

Searching indexed records.

Library Catalogs

Fast retrieval.

Interview Purpose

Tests:

Binary Search

Boundary Handling

Search Space Reduction

14. Common Traps (Important!)

Trap 1

Forgetting array must be sorted.

Trap 2

Infinite loops.

Wrong:

left = mid

or

right = mid

Trap 3

Integer overflow in Java/C++.

Use:

$\text{left} + (\text{right} - \text{left}) / 2$

instead of:

$(\text{left} + \text{right}) / 2$

15. Builds To (Related LeetCode Problems)

LC 704 - Binary Search



LC 35 - Search Insert Position



LC 69 - Sqrt(x)



LC 278 - First Bad Version



LC 367 - Valid Perfect Square



LC 33 - Search in Rotated Sorted Array

These are foundational Binary Search problems.

16. Alternate Approaches + Comparison

Approach	Time	Space	Interview Rating
Linear Search	$O(n)$	$O(1)$	Poor
Binary Search	$O(\log n)$	$O(1)$	Best

Approach 1: Linear Search

Check every element.

Simple but slow.

Approach 2: Binary Search (Chosen)

Uses sorted order.

Much faster.

17. Why This Solution Works (Short Intuition)

Because the array is sorted, whenever we compare the target with the middle element, we can eliminate half of the remaining search space.

Repeating this process quickly narrows the search to the target or proves it doesn't exist.

18. Variations / Follow-Up Questions

Follow-Up 1

Find insertion position.

Related:

LC 35

Search Insert Position

Follow-Up 2



Find first occurrence of a duplicate.

Example:

[1,2,2,2,3]

Return:

1

Follow-Up 3

Find last occurrence.

Example:

[1,2,2,2,3]

Return:

3

Follow-Up 4

Binary Search on Answer Space.

Examples:

LC 69 Sqrt(x)

LC 875 Koko Eating Bananas

LC 1011 Capacity To Ship Packages

Interview Takeaway

This is the most important Binary Search problem.

Core idea:

Sorted Data

+

Eliminate Half Every Step

Whenever you hear:

Sorted Array

Fast Search

Find Position

immediately think:

Binary Search

This pattern appears repeatedly in product-company interviews, competitive programming, and advanced optimization problems.